

# Aufgabenblatt 7

## Minimale Spannbäume

### Abgabe (bis 30.06.2025 23:59 Uhr)

Die folgenden Dateien werden für die Bepunktung berücksichtigt:

Blatt07/src/EdgeWeightedGraph.java Aufgabe 3.1

Blatt07/src/Clustering.java Aufgabe 3.2 bis 3.5

Als Abgabe wird jeweils nur die letzte Version im main branch in git gewertet.

### Aufgabe 1: Union Find (Tutorium)

**1.1** Was ist eine Äquivalenzrelation? Was sind Äquivalenzklassen und wie hängen diese mit Zusammenhangskomponenten zusammen?

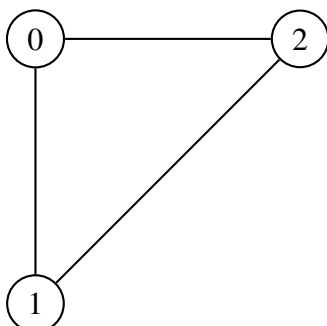
#### Lösung

Äquivalenzrelation:

- **reflexiv:** Ein Objekt steht mit sich selbst in Beziehung
- **symmetrisch:** Wenn ein Objekt  $A$  mit Objekt  $B$  in Beziehung steht, dann steht auch  $B$  mit  $A$  in Beziehung
- **transitiv:** Wenn  $A$  mit  $B$  in Beziehung steht und  $B$  mit  $C$ , dann steht auch  $A$  mit  $C$  in Beziehung

Eine Äquivalenzrelation teilt eine Menge in disjunkte Äquivalenzklassen. In einem Graphen ist die ungerichtete Verbindung zwischen Knoten eine Äquivalenzrelation; die Knoten sind dann zusammenhängend. Die Äquivalenzklassen dazu sind dann die Zusammenhangskomponenten.

**1.2** Wozu wird Union Find benutzt? In welchem Algorithmus spielt er eine Rolle? Wie funktioniert Union Find? Gehen Sie den Algorithmus schrittweise an folgendem Beispiel durch:



### Lösung

Der *Union Find* findet Zusammenhangskomponenten und damit z.B. auch Zyklen in Graphen. Er wird u.a. im *Kruskal*-Algorithmus verwendet.

*Union Find* bietet eine Struktur zur Speicherung dynamischer Zusammenhangskomponenten. Dabei kann mit der `find()`-Methode die Zusammenhangskomponente eines Knoten ausgegeben werden. Mit `union(int v, int w)` werden die Zusammenhangskomponenten der Knoten `v` und `w` verbunden, falls sie nicht bereits verbunden sind. Die Methode `connected(int v, int w)` gibt zurück, ob zwei Knoten in der gleichen Zusammenhangskomponente sind.

Wenn wir den obigen Graphen in den Algorithmus eingeben, würden wir also die `union`-Methode nutzen:

1. Konstruktoraufwurf: `UnionFind(3)` gibt 3 Zusammenhangskomponenten für jeweils eine für die Knoten 0, 1 und 2
2. `union(0, 1)` - verbindet 0 und 1. Achtung! Je nach Union Find Algorithmus, also quick find, quick union oder eager union, wird diese Verbindung anders gespeichert
  - Knoten-indiziertes Array `id` speichert die Nummer einer Zusammenhangskomponenten
  - Elternknoten-Array `parent` (in `find`, muss nun nach der Wurzel gesucht werden)
3. `connected(0, 1)` gibt nun `true` zurück
4. `union(0, 2)` - verbindet A und C
5. `connected(1, 2)` gibt jetzt auch `true` zurück.

1.3 Funktioniert der folgende Code-Abschnitt für Union Find? Warum wäre das der Fall?

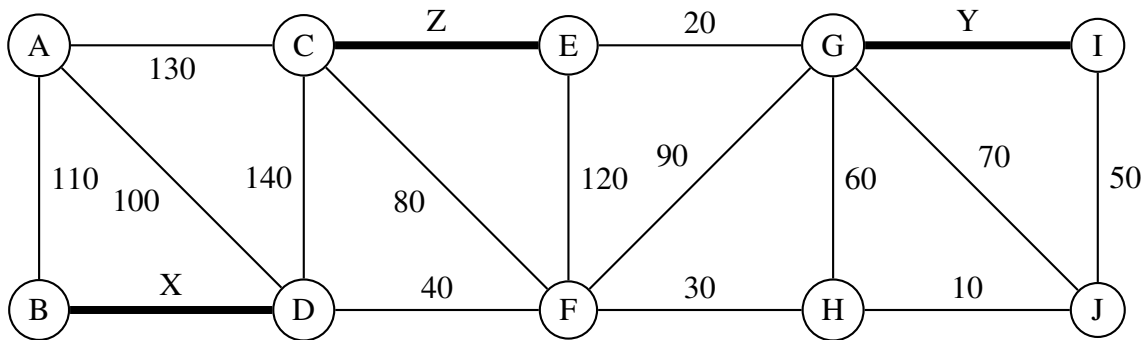
```
public void union(int p, int q) {  
    if (connected(p, q)) return;  
    for (int i = 0; i < id.length; i++)  
        if (id[i] == id[p]) id[i] = id[q];  
    count--;  
}
```

### Lösung

Das ist der quick find Ansatz. Der Abschnitt funktioniert nicht, der Wert `id[p]` wird auf `id[q]` gesetzt in der for-Schleife. Deshalb wird jeder Knoten `r > p` mit `id[r] == id[p]` nicht auf `id[q]` angepasst.

## Aufgabe 2: Minimum Spanning Tree (Tutorium)

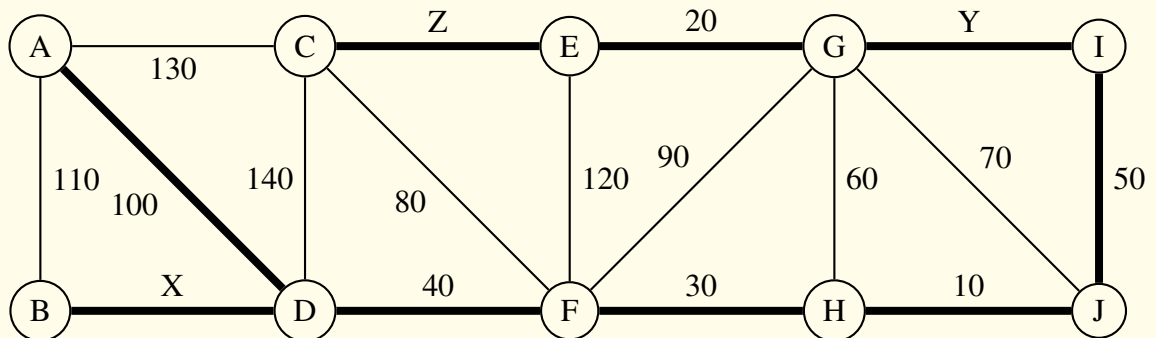
Nehmen Sie an, der Minimum Spanning Tree (MST) dieses Graphen enthalte die Kanten X, Y und Z.



- 2.1 Notieren Sie die Gewichte der restlichen Kanten, d.h. alle außer X, Y und Z, die zum MST dieses Graphen gehören müssen unter der obigen Annahme.

### Lösung

Kanten nach weights sortiert:  
10, 20, 30, 40, 50, 100



- 2.2 Geben Sie jeweils die größte obere Schranke für die Gewichte der Kanten X, Y und Z an, mit der garantiert ist, das alle drei Kanten tatsächlich Teil des MST sind.  
Hinweis: Geben Sie individuell für jedes Gewicht x, y, z eine Schranke an, wobei x, y und z die entsprechenden Gewichte zu den Kanten X, Y und Z sind.

### Lösung

Eigentlich ein  $<$ , aber  $\leq$  wird auch akzeptiert.

$$x < 110$$

$$y < 60$$

$$z < 80$$

2.3 Setzen Sie für X, Y und Z folgende Gewichte ein und führen Sie Kruskal und Prim ab A aus.

$$x = 120$$

$$y = 50$$

$$z = 80$$

### Lösung

Kruskal: Kanten werden aufsteigend sortiert angeschaut. Wenn sie keinen Kreis erzeugen, werden sie eingefügt.

Kanten in der Reihenfolge des Hinzufügens:

$[(H, J)^{10}, (E, G)^{20}, (F, H)^{30}, (D, F)^{40}, (G, I)^{50}, (I, J)^{50}, (C, E)^{80}, (A, D)^{100}, (A, B)^{110}]$

Prim: Ausgehend von einem Startknoten, wird die nächst-kleinste kreuzende Kante hinzugefügt. Der Begriff *kreuzende Kante* wird ausführlich im Skript auf Seite 145 beschrieben.

Kanten in der Reihenfolge des Hinzufügens:

$[(A, D)^{100}, (D, F)^{40}, (F, H)^{30}, (H, J)^{10}, (J, I)^{50}, (I, G)^{50}, (G, E)^{20}, (C, E)^{80}, (A, B)^{110}]$

Statt der Kante  $(C, E)^{80}$  kann in beiden Fällen auch die Kante  $(C, F)^{80}$  gewählt werden.

### **Aufgabe 3:** Für die Hausaufgaben wird keine Lösung bereitgestellt.

#### **Was Sie nach diesem Blatt wissen sollten:**

- Was Union Find ist und wie es funktioniert
- Was ein MST ist und welche Eigenschaften er hat
- Wofür man einen MST nutzen kann
- Was ein Schnitt ist und wie die Schnitteigenschaft definiert ist
- Was kreuzende Kanten sind
- Was das allgemeine Vorgehen zum Bestimmen eines MST ist
- Wie der *Prim*- und der *Kruskal*-Algorithmus funktionieren
- Was die beiden Algorithmen unterscheidet